T01.S02 — The Real-Time Constraint

Topic: T01 — Real-Time Programming Foundations | **Depends on:** T01.S01 **Status:** expanded | **Assignments:** A001 | **Est. theory time:** 2 sessions

> The question this subtopic answers: *what does "runs at 60 FPS" actually commit you to, and why does the number everyone quotes — average FPS — hide the thing players actually feel?*

Theory

U01 — The frame budget

What it is. A fixed wall-clock allowance per frame, set by the target refresh rate, that all work must fit inside.

Why it exists. The display refreshes at a fixed rate. If your frame isn't ready when it refreshes, the player sees the previous frame again — a stutter. There is no "finish a bit late and it's fine".

The mechanism.

target	budget	what it means
30 Hz	33.3 ms	Console-cinematic; noticeably laggy for action
60 Hz	16.67 ms	The standard bar
120 Hz	8.33 ms	High-refresh desktop
144 Hz	6.94 ms	Common gaming monitor
90 Hz per eye	11.1 ms	VR minimum; missing it causes actual nausea

That budget is not yours alone. A plausible 16.67 ms breakdown on the CPU side of a mid-size game:

```
  1.5 ms  input + OS event pump + engine bookkeeping
  3.0 ms  gameplay + scripts + AI
  3.5 ms  physics (may be multiple sub-steps)
  1.5 ms  animation sampling + skinning setup
  2.5 ms  culling + render list building + draw submission
  1.0 ms  audio update (mixing itself is on another thread)
  ------
 13.0 ms  used
  3.7 ms  headroom  <-- this is not spare, it is the margin for the
                        worst frame, not the average one
```

The GPU is doing its own work concurrently against the same 16.67 ms. Either can be the bottleneck; "the game is slow" is not a diagnosis (`T13.S02`).

Where it bites. People budget for the average frame and ship a game that hitches whenever something unusual happens — a particle burst, an asset load, a GC-like batch free. The headroom line above is the whole game.

Read.

- `[GEA3]` ch. 8.5 — the real-time simulation section.
- `[RTR4]` ch. 1 — short, and frames the CPU/GPU concurrency point.

Check yourself.

1. What is the frame budget at 144 Hz, and how much of it does a single 4 ms physics step consume?
2. Why is CPU frame time not simply "sum of subsystem times"?

U02 — The latency chain: input to photon

What it is. The total delay between a physical action by the player and the corresponding light leaving the monitor. It is much longer than one frame, and almost none of it is your game logic.

Why it exists. Every stage in the chain buffers. Buffers exist for throughput and smoothness; each one costs latency.

The mechanism. A realistic chain at 60 Hz with a typical setup:

```
physical press
  |  ~1-8 ms    USB polling interval (125 Hz default = 8 ms; 1000 Hz = 1 ms)
  v
OS input queue
  |  0-16.7 ms  your loop polls once per frame; the press could have
  |             happened at any point since the last poll
  v
simulation consumes it
  |  ~16.7 ms   the frame is simulated and submitted
  v
GPU renders
  |  ~16.7 ms   GPU works on frame N while CPU builds N+1
  v
present queue / swap chain
  |  0-33 ms    1-3 frames deep depending on driver settings
  v
display scanout + pixel response
  |  ~5-15 ms   panel refresh and pixel transition
  v
photon
```

Realistic total: **50–100 ms** at 60 Hz on default settings. Competitive setups get it near 20–30 ms by raising polling rate, reducing queue depth, and running at high refresh.

Two things follow that are not obvious:

- **Reducing frame time reduces latency more than proportionally**, because several stages are quantized to frame boundaries.
- **A frame of latency you add is invisible in your FPS counter.** Double buffering the input, or updating the camera a frame behind the player, costs 16.7 ms and shows up in no metric except how the game feels.

Where it bites. The classic self-inflicted frame: reading input *after* the simulation update, so a press is acted on next frame. Free to fix, easy to introduce, and the resulting mush is usually blamed on "the engine".

Read.

- `[GAMEFEEL]` ch. 3–4 — the perceptual side: how much latency players can detect, which is less than you think.
- NVIDIA's Reflex / latency articles — https://developer.nvidia.com/blog/reflex-low-latency-platform/ — for the present-queue portion of the chain, described by the people who own that stage.

Check yourself.

1. List every buffer between a keypress and a photon, and give an approximate cost for each.
2. Your game runs at a solid 60 FPS and feels laggy. Name three causes that would not change the FPS number.

U03 — Vsync, tearing, and variable refresh

What it is. The set of policies for deciding when a finished frame is handed to the display, and what happens when it isn't finished in time.

Why it exists. The display scans out pixels top to bottom on a fixed clock. If the framebuffer changes mid-scanout, the top and bottom of the screen show different frames — a tear.

The mechanism.

- **Vsync off** — present immediately. Lowest latency, visible tearing.
- **Vsync on (double buffered)** — present only at the vertical blank. No tearing. If you miss the deadline by 0.1 ms, you wait a *whole* refresh: effective rate drops 60 → 30. This cliff is why vsync feels so bad when you are marginal.
- **Triple buffering** — a third buffer absorbs the miss, so a late frame costs less than a full refresh, at the cost of another frame of latency.
- **Adaptive vsync** — vsync when above target, off when below (trading tearing for the cliff).
- **VRR (G-Sync / FreeSync / VESA Adaptive-Sync)** — the *display* waits for the frame instead. Removes both the tear and the cliff within the panel's supported range. This is now the common case on gaming hardware and it changes the calculus: framerate consistency still matters, but the 60→30 cliff largely doesn't.

Where it bites. Benchmarking with vsync on. Your loop is now measuring the display, not your engine — every frame reads as exactly 16.67 ms and your profiling data is worthless. `A001` requires vsync off for exactly this reason.

Read.

- `[SDLDOCS]` — `SDL_GL_SetSwapInterval` / `SDL_SetRenderVSync`, and read what the return values mean on each backend.
- `[RTR4]` §23.6 — buffering and the swap chain from the hardware side.

Check yourself.

1. Explain the 60→30 cliff precisely: what is the program doing during the lost time?
2. Why does VRR not eliminate the need for consistent frame times?

U04 — Averages lie: frame time distributions

What it is. The measurement discipline: report frame *times* in milliseconds as a distribution, never framerate as a mean.

Why it exists. Three reasons, and each is independently sufficient:

1. **FPS is a reciprocal, so averaging it is wrong.** Averaging 30 FPS and 60 FPS gives 45 FPS, but the actual time taken is 33.3 + 16.7 = 50 ms for two frames = 40 FPS. Means of reciprocals are not reciprocals of means.
2. **A mean hides the spikes**, and spikes are what players perceive. A run where every frame is 16 ms and a run averaging the same value but containing occasional 60 ms frames feel completely different; the second one is described as “stuttering” and the first as “smooth”.
3. **Milliseconds are linear and additive**, so you can reason about budgets with them. “I saved 2 ms” is meaningful. “I gained 10 FPS” is meaningless without knowing where you started — 10 FPS is 5.5 ms at 30 FPS and 0.9 ms at 120 FPS.

The mechanism. Report, at minimum: **mean, 95th percentile, 99th percentile, and max**, plus a histogram. The industry has largely converged on **1% low** and **0.1% low** framerates in reviews, which are the same idea (the mean of the worst 1% of frames) expressed for consumers.

```
frame time histogram, 3600 frames, vsync off
ms     count
 8-10  |================================== 1820
10-12  |======================= 1190
12-14  |========= 460
14-16  |=== 98
16-20  |= 24
20-40  | 6
40+    | 2     <-- these eight frames are the entire user experience
                   of "it stutters sometimes"
```

Where it bites. Optimizing the mean. Shaving 0.5 ms off a function that runs every frame while ignoring the 40 ms asset-load hitch is a common and completely wasted week. Percentiles direct you at the right target.

Read.

- Gil Tene, "How NOT to Measure Latency" — https://www.youtube.com/watch?v=lJ8ydIuPFeU — not games-specific, and the best possible treatment of why averages and coordinated omission mislead. Watch this one.
- `[GEA3]` ch. 8.5.3 — measuring and dealing with frame time.

Check yourself.

1. Frames take 10, 10, 10, 10, 50 ms. What is the mean FPS, and what is the average of the per-frame FPS values? Explain the discrepancy.
2. Your p50 improved and your p99 got worse. Is that a win? Under what circumstances?

U05 — Determinism and reproducibility

What it is. The property that identical inputs produce an identical sequence of states. Not free, not automatic, and required by several later topics.

Why it exists. It buys you: replays (`A021`), rollback netcode (`A047`), reproducible bug reports, and automated regression testing of gameplay. Very few of these are retrofittable.

The mechanism. Sources of nondeterminism, in the order they will bite:

- **Variable timestep.** If `dt` differs between runs, everything differs. Fixed timestep (`T03.S01`) is the precondition for all the rest.
- **Uninitialized memory / reading stale state.** Nondeterministic in exactly the way that makes bugs unreproducible.
- **Iteration order over unordered containers.** `std::unordered_map` order depends on insertion history and, across builds, on the hash. Any physics or AI that iterates one is nondeterministic.
- **Pointer values used as keys or sort tie-breakers.** ASLR makes these differ per run.
- **Floating point across compilers/platforms/optimization levels.** Within a single binary on a single machine, FP is deterministic. Across them: `-ffast-math`, FMA contraction, and x87 80-bit intermediates break it. This is the reason cross-platform lockstep is hard.
- **Multithreaded accumulation.** Float addition is not associative, so summing in a nondeterministic order gives different results (`T13.S05`).
- **Time or RNG read outside the simulation.** Seed the RNG from the simulation state, and never read wall-clock inside gameplay.

Where it bites. You will not discover you lack determinism until `A021` or `A047`, by which point it is an archaeology project. Cheap insurance now: fixed timestep, ordered containers in simulation code, a seeded RNG owned by the sim, and a `state hash` per frame you can compare between two runs of the same input.

Read.

- `[GAFFER]` “Deterministic Lockstep” — https://gafferongames.com/post/deterministic_lockstep/ — the requirements listed by someone who has paid for each one.
- `[GOLDBERG]` — skim §1–2 now for the float model; you will return to it.

Check yourself.

1. Your game desyncs between two machines but replays fine on each. Which category of cause is that, and which is ruled out?
2. Why is `std::unordered_map` iteration a determinism bug but `std::map` iteration not?

Implementation

assignment	size	target rung	what it forces you to understand
A001	XS	R2	That you cannot describe performance with one number; and how to take a timing measurement that is not itself the thing you are measuring

Order and overlap. `A001` is the first thing you write in this track and it is deliberately tiny. Its output — the percentile reporter — gets copied into `A002`’s window skeleton, then becomes the debug overlay in `A006`, then grows into the real frame profiler in `A042`. You are writing the first version of a tool you will use for two years.

If you are short on time. `A001` is one hour. There is no shorter version. Do it.

Discussion Log

Open Questions

Connections

- `T01.S04` — how to take the measurements this unit says you need.
- `T03.S01` — the fixed timestep that U05’s determinism depends on.
- `T13.S02` — the grown-up version of `A001`.
- `T14.S05` — where U05’s determinism requirement is finally cashed in.